

Свёрточные нейронные сети

Свёрточные нейронные сети (СНС или CNN) — это особый вид искусственных нейронных сетей, предназначенный для эффективного распознавания образов, особенно изображений. Они основаны на чередовании свёрточных слоев и слоев подвыборки (пулинга) и используют принципы, напоминающие работу зрительной коры человека.

Основной принцип работы свёрточной сети — применение небольших фильтров (ядер свёртки), которые последовательно сканируют входное изображение или карту признаков. Каждый фильтр — это небольшая матрица весов, которая умножается на фрагменты данных, суммируются результаты, и формируется новая карта признаков, отражающая наличие и расположение определённых признаков (например, линий, углов) на изображении.

В отличие от полносвязных сетей, где каждый нейрон связан со всеми входами, в свёрточных сетях фильтры повторно применяются по всему полю зрения, что существенно снижает количество обучаемых параметров и учитывает локальную структуру данных. Благодаря этому СНС показывают высокую устойчивость к смещениям, масштабированию и вращению объектов на изображении.

Типичная архитектура СНС включает:

Свёрточные слои для выделения признаков,

Слои пулинга для уменьшения размерности и выделения важных признаков,

Слои нормализации,

Полносвязные слои для классификации.

Свёрточная нейронная сеть строится по принципу пирамиды — первые слои содержат больше нейронов, а последующие — всё меньше и меньше. Связано это с тем, что низкоабстрактных признаков больше, чем высокоабстрактных.

Как правило, на конце нейронной сети располагаются несколько полносвязных слоёв. Эти слои учатся на немногочисленных высокоабстрактных признаках, не требующих много слоёв, что считается приемлемым с точки зрения вычислительных затрат. Получается, что свёрточную нейронную сеть можно условно поделить на две части — одна извлекает признаки, а другая, полносвязная, на них обучается.

Ниже пример простой реализации свёрточной нейронной сети (CNN) на Keras для задачи классификации изображений (например, на наборе данных MNIST):

```
import tensorflow as tf
from tensorflow.keras import layers, models

# Создаем модель
model = models.Sequential([
```

```

layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28,
    28, 1)), # Свёрточный слой
layers.MaxPooling2D((2, 2)), # Пулинг слой для уменьшения
    размерности
layers.Conv2D(64, (3, 3), activation='relu'), # Второй
    свёрточный слой
layers.MaxPooling2D((2, 2)),
layers.Flatten(), # Преобразуем 2D-данные в 1D вектор
layers.Dense(64, activation='relu'), # Полносвязный слой
layers.Dense(10, activation='softmax') # Выходной слой для 10
    классов
])

# Компилируем модель
model.compile(optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])

# Краткий обзор модели
model.summary()

```

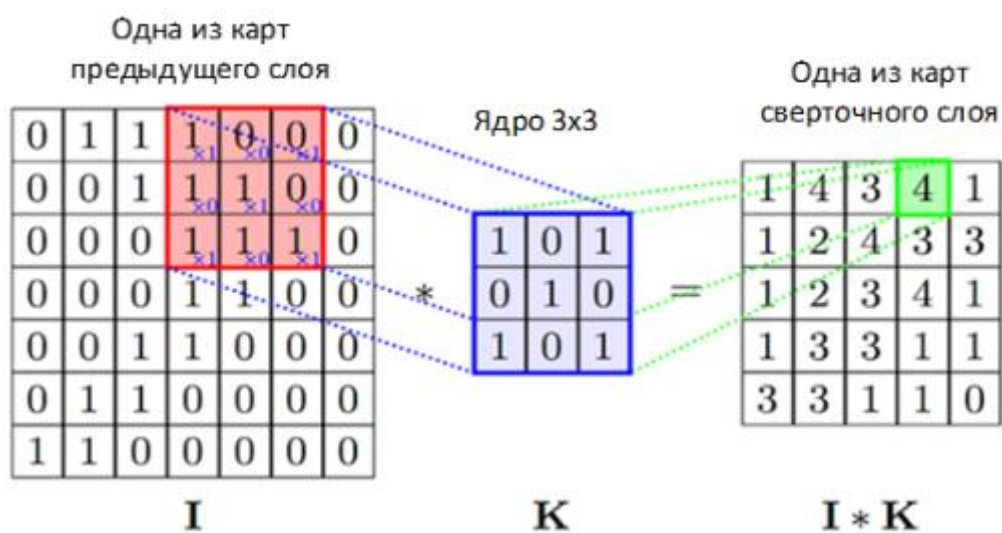
Этот код создает модель с двумя свёрточными и пулинг-слоями, затем данные выравниваются и проходят через полносвязные слои, после чего выдаются вероятности принадлежности к 10 классам.

Для обучения достаточно вызвать:

```
model.fit(x_train, y_train, epochs=5, batch_size=32)
```

где `x_train` — тренировочные изображения, а `y_train` — метки классов. Для MNIST размер изображений 28x28, и они должны быть в формате (28, 28, 1) с нормализацией пикселей.

В свёрточных нейронных сетях свёртка — это операция, суть которой в последовательном скольжении небольшой матрицы весов — ядра свёртки (кernels) — по входному изображению или карте признаков. На каждом шаге ядро поэлементно умножается на соответствующую часть входных данных, а результаты суммируются, формируя один элемент выходной карты признаков.

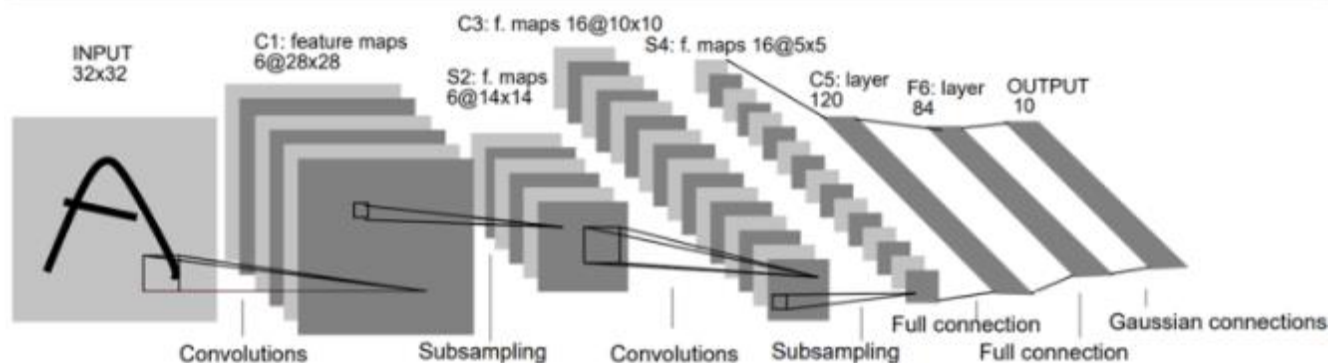


Проще говоря, ядро — это фильтр, который "прислушивается" к определённым признакам изображения, например, к краям или текстурам. По мере обучения сеть самостоятельно подбирает веса ядра, чтобы обнаруживать полезные для задачи характеристики. Благодаря использованию одного и того же ядра во всех позициях данных, свёртка позволяет выявлять признаки независимо от их расположения в изображении, снижая при этом число параметров по сравнению с обычными полносвязными слоями.

Результатом работы свёртки является карта признаков — двумерный массив, показывающий, насколько в каждой части входного изображения обнаружен данный признак. Множество таких ядер создаёт множество карт признаков, что позволяет сети строить иерархию от простых элементов к сложным образам.

Нейронная сеть Lenet5

Lenet5 — одна из первых свёрточных нейронных сетей, она отражает характерные для них набор элементов: свёрточные слои, пуллинг слои и полносвязные слои на конце нейронной сети. Данная архитектура послужила основой для многих современных архитектур свёрточных нейронных сетей.



LeNet-5 — это классическая свёрточная нейронная сеть, разработанная Яном ЛеКуном в 1998 году для распознавания рукописных цифр. Архитектура сети

включает последовательность из трех свёрточных слоев и двух полносвязных слоев, что и отражено в названии LeNet-5 (5 обучаемых слоев).

Основные слои и их функции:

C1 — первый свёрточный слой с 6 картами признаков, размер ядер 5x5, активация tanh.

S2 — слой подвыборки (average pooling) с ядром 2x2 для уменьшения размерности.

C3 — второй свёрточный слой с 16 картами признаков, ядро 5x5.

S4 — второй подвыборочный слой с ядром 2x2.

C5 — третий свёрточный слой с 120 картами признаков, ядро 5x5, результат подается в полносвязные слои.

F6 — полносвязный слой с 84 нейронами.

Выходной слой с 10 нейронами для классификации (например, цифр 0-9).

Входные данные — изображения размером 32×32 пикселя с одним каналом (оттенки серого). Важной особенностью LeNet-5 является использование слоев усредняющей подвыборки и гиперболического тангенса (tanh) как функции активации, что было характерно для тех лет.

LeNet-5 стала одной из первых эффективных архитектур для задач компьютерного зрения, заложив фундамент для современных свёрточных сетей. Она сочетает выделение пространственных признаков свёрточными слоями с последующей классификацией на полносвязных слоях, что позволяет эффективно распознавать образцы с сохранением локальной структуры изображения.

Пример реализации LeNet-5 на Keras для MNIST

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.datasets import mnist
import numpy as np

# Загрузка данных MNIST
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Расширение размера изображений до 32x32 и добавление канала
x_train = np.pad(x_train, ((0,0),(2,2),(2,2)), mode='constant')
x_test = np.pad(x_test, ((0,0),(2,2),(2,2)), mode='constant')
x_train = x_train.reshape(-1, 32, 32, 1).astype('float32') / 255.0
x_test = x_test.reshape(-1, 32, 32, 1).astype('float32') / 255.0

# Категориальная кодировка меток
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Построение модели LeNet-5
model = models.Sequential([
```

```

        layers.Conv2D(6, kernel_size=5, activation='tanh',
input_shape=(32, 32, 1)),
        layers.AveragePooling2D(),
        layers.Conv2D(16, kernel_size=5, activation='tanh'),
        layers.AveragePooling2D(),
        layers.Conv2D(120, kernel_size=5, activation='tanh'),
        layers.Flatten(),
        layers.Dense(84, activation='tanh'),
        layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Обучение модели
model.fit(x_train, y_train, epochs=10, batch_size=128,
validation_split=0.1)

# Оценка качества на тесте
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f'Test accuracy: {test_acc:.4f}')

```